

03 ドメインモデル

03. ドメインモデル（ドラフト）

- ドキュメント種別: 上流工程 / ドメインモデル（概念モデル・ER・状態遷移）
- 対象システム（仮称）: 居酒屋店舗システム（SaaS型）／AIネイティブ再構築版
- 作成日: 2026-09-03
- ステータス: フェーズ1向け凍結（2026-09-09）。第7章の未決事項は全件解決済み（残課題はフェーズ1スコープ外または実装スパイク送り）。以降の変更はフェーズ1スコープ内の誤り訂正・実装スパイク結果の反映に限る。
- 関連文書: 01_system_overview.md、02_requirements.md（本書は02のフェーズ1要件に基づく）

本書はフェーズ1の要件（02_requirements.md 第5章 FR-A～FR-K、および第2.5節の決定事項）に基づいて、主要エンティティ、ER図（Mermaid）、主要な状態遷移（Mermaid）を定義する。物理テーブル定義・カラム型・インデックス・APIは04_architecture.mdで確定する。

Mermaid 記法について: 本書の図は ``mermaid`` フェンスで記述している。GitHub / VS Code / Mermaid 対応ビューアでは図として表示される。pandoc + wkhtmltopdf での PDF 化ではコードのまま 出力されるため、PDF に図を含めたい場合は Mermaid のプリレンダリングが必要（04` で方針を決める）。

1. モデリング方針（02 2.5 の決定を反映）

#	方針	根拠
1	マルチテナント：全業務テーブルはテナント識別子 company_code を持つ（共有DB・共有スキーマ）。アプリ層で必ずテナント境界を強制する。	2.5 マルチテナント分離、FR-A06
2	店舗スコープ：全業務テーブルは store_id を持つ。フェーズ1は「1テナント1店舗」だが、構造は複数店舗前提。	2.5 店舗数、NFR-07
3	金額は整数（円）。amount_jpy のように単位を明示。丸め規則は税計算で定義（04）。	DAT-03
4	価格・名称はスナップショット：注文明細・会計明細は、その時点のメニュー名・単価・税区分を複製して保持する（後からメニューを変えても過去伝票は不変）。	FR-K01、FR-D01
5	確定後は不変・訂正はイベント：会計確定後の修正は「返金」、日次締め後の修正は「翌営業日の補正」。元レコードは削除しない。	FR-G10、FR-H01
6	イベントの記録：卓・注文明細・会計の状態変化を時刻付きイベントとして domain_event に残す（分析・AIの後付けのため）。	FR-K02
7	監査ログ：重要操作は audit_log に追記専用で記録。業務テーブルとは分離。	FR-J01～J05
8	論理削除：マスタ（メニュー、卓、スタッフ等）は物理削除せず is_active で無効化（過去伝票の参照整合のため）。	FR-K01
9	共通監査項目：全テーブルに created_at / updated_at、業務テーブルに created_by / updated_by（既存 BaseEntity を踏襲・拡張）。	既存実装
10	長期保持：取引・帳簿系（注文明細、会計、決済、返金、日次締め、domain_event）は長期保持前提（7～10年目安、確定は法務）。	2.5 データ保持方針、DAT-04

2. 主要エンティティ一覧（フェーズ1）

区分の凡例：M=フェーズ1で必須、S=可能なら（難ければフェーズ2）、（基盤）=分析・AIの後付け用。

属性名は物理カラム名（英字 snake_case、方針 #6・第6章）を正とし、意味が分かりにくい語には 属性名（和名）の形で日本語を併記する（id / name / created_at 等の自明な語は併記しない）。

2.1 テナント・組織・ユーザー

エンティティ	区分	目的	主な属性（代表）
company（テナント）	M	契約単位。1オーナー＝1テナント	id, company_code（テナント識別子, UK）, name, contract_status（契約状態）, created_at
store（店舗）	M	営業拠点。全業務データのスコープ	id, company_id（テナント識別子, FK）, name, address, phone, business_hours（営業時間）, seat_count（座席数）, timezone（タイムゾーン）, is_active（有効フラグ）
store_setting（店舗設定）	M	税・予約・モバイルオーダー等の店舗ポリシー（store と 1:1）	store_id, tax_rounding（税額丸め規則）, price_includes_tax（税込価格か）, invoice_reg_no（適格請求書登録番号）, web_reservation_mode（Web予約受付方式, APPROVAL/INSTANT）, mobile_order_enabled（モバイルオーダー有効）, last_order_default_min（ラストオーダー既定, 分）, cancel_charge_default_customer（客都合キャンセルの既定課金, bool）, cancel_charge_default_store（店都合キャンセルの既定課金, bool）
user（利用者）	M	ログインユーザー。既存 users を継承（company_id + email 複合UK）	id, company_id（テナント識別子, FK）, store_id（所属店舗, nullable=全店）, name, email, password(hash), telnumber（電話番号）, role（権限ロール, OWNER/MANAGER/HALL/KITCHEN/PARTTIME）, status（アカウント状態, ACTIVE/LOCKED/INVITED）, two_factor_enabled（2要素認証有効）
user_invitation（招待）	M	招待リンクの発行・失効	id, company_id（テナント識別子, FK）, store_id, email, role, token（招待トークン）, expires_at（有効期限）, accepted_at（受諾日時）

company を新設し id（サロゲートキー）を主キーとする。company から1ホップの直下テーブル（store・user・user_invitation）は実FKの company_id を持つ（詳細は第7章1・04_architecture.md）。それ以外の業務テーブルは、テナント絞り込み用に company_code 列（FK制約なしの非正規化コピー）を保持する。

2.2 マスタ（メニュー・卓・コース・決済手段）

エンティティ	区分	目的	主な属性（代表）
menu_category（メニューカテゴリ）	M	刺身／揚げ物／ドリンク等	id, store_id, name, display_order（表示順）, is_active（有効フラグ）

menu_item（メニュー項目）	M	品目	id, store_id, category_id, prep_type（調理要否, COOK/NO_COOK, NO_COOK=ドリンク等は調理KDS/ kitchen.ticket の対象外。04 59）, name, description, price_jpy（単価, 円）, tax_category（税区分, STANDARD_10/REDUCED_8）, photo_url, serve_time_from（提供時間帯 開始）, serve_time_to（提供時間帯 終了）, available_from（提供期間 開始）, available_to（提供期間 終了, 期間限定）, sales_status（販売状態, ON_SALE/SOLD_OUT/SUSPENDED）, display_order（表示順）, is_active（有効フラグ）
menu_option_group（オプション群）	S	「サイズ」「トッピング」等	id, store_id, menu_item_id(または共有), name, min_select（最小選択数）, max_select（最大選択数）
menu_option（オプション）	S	「大盛り+150円」等	id, option_group_id, name, price_delta_jpy（追加料金, 円）, is_active（有効フラグ）
course（コース・飲み放題）	S	時間管理の対象	id, store_id, name, type（種別, COURSE/FREE_DRINK）, duration_min（制限時間, 分）, last_order_before_min（終了前ラストオーダー, 分）, price_jpy（料金, 円）, is_active（有効フラグ）
dining_table（卓）	M	客席。モバイルオーダーのQR紐付け先	id, store_id, table_no（卓番号）, seat_count（座席数）, area（エリア）, qr_token（QRトークン, 店内UK）, status（卓状態, EMPTY/OCCUPIED/BILLING＝一覧表示用の導出値）, is_active（有効フラグ）
payment_method_config（決済手段設定）	M	店舗が有効化した決済手段	id, store_id, method_type（決済手段種別, CASH/PAYPAY/CREDIT_CARD/RAKUTEN_PAY）, enabled（有効）, display_name（表示名）, credential_enc（認証情報, 暗号化, CASH/RAKUTEN_PAYは空可）, note
store_business_day（営業日設定）	M	営業日／臨時休業。Web予約受付可否に反映	id, store_id, date_or_weekday（日付 or 曜日）, is_open（営業するか）, open_time（開店時刻）, close_time（閉店時刻）, reservation_capacity（予約受入可能数）

2.3 予約

エンティティ	区分	目的	主な属性（代表）
reservation（予約）	M	電話／Web／当日ウォークインを一元管理	id, store_id, reserved_at（予約日時）, party_size（人数）, guest_name, guest_phone, guest_email, request_note（アレルギー等）, course_id(nullable), channel（予約経路, WEB/PHONE/WALK_IN）, status（予約状態, REQUESTED/CONFIRMED/SEATED/DONE/CANCELLED/NO_SHOW）, created_by(nullable=Web), confirmed_by（確定者）, confirmed_at（確定日時）, cancelled_reason（キャンセル理由）
reservation_notification（予約通知）	M	受付確認・前日リマインダの送信記録	id, reservation_id, type（通知種別, CONFIRM/REMINDER）, channel（送信手段, EMAIL/SMS）, to（宛先）, status（送信状態, QUEUED/SENT/FAILED）, sent_at（送信日時）

2.4 来店・卓セッション・注文

エンティティ	区分	目的	主な属性（代表）
table_session（卓セッション＝来店）	M	卓のオープンからクローズまでの一連の飲食	id, store_id, status（セッション状態, OPEN/BILLING/CLOSED）, opened_at（開始日時）, opened_by（開始操作者）, closed_at（終了日時）, party_size（人数）, reservation_id(nullable), course_id(nullable), course_started_at（コース開始日時）, last_order_at（ラストオーダー時刻）
table_session_table（セッション中卓）	S	卓の結合。1セッションが複数卓を占有	table_session_id, dining_table_id, is_primary（主卓か）
mobile_order_session（モバイルオーダーセッション）	M	卓上QRから開く未ログインの注文セッション	id, table_session_id, qr_token（QRトークン）, issued_at（発行日時）, expires_at（有効期限）, status（ACTIVE/EXPIRED）(卓クローズで EXPIRED)
order（注文＝1回の送信）	M	スタッフ入力 or モバイル送信の単位	id, table_session_id, dining_table_id（注文時点の物理卓, FK→dining_table）, source（注文元, STAFF/MOBILE）, entered_by（入力者, user, nullable）, mobile_order_session_id(nullable), status（注文状態, SUBMITTED/ACCEPTED/REJECTED）, submitted_at（送信日時）, accepted_by（受理者）, accepted_at（受理日時）, reject_reason（却下理由）
order_line（注文明細）	M	品目単位。分析の最小粒度	id, order_id, table_session_id, menu_item_id, item_name_snap（品名スナップショット）, unit_price_snap_jpy（単価スナップショット, 円）, tax_category_snap（税区分スナップショット）, quantity（数量）, note, serve_status（提供状態, PENDING/PREPARING/SERVED/CANCELLED/REJECTED）, fire_state（調理投入状態, HELD/FIRED。既定 FIRED。HELD＝後出し保留で調理KDS非表示、ホールの fire で FIRED。04 59）, fired_at（fire 日時。HELD を経た明細は KDS のソート・滞留をこれで測る）, fired_by（fire 操作者）, registered_at（登録日時）, registered_by（登録者）, business_date（注文された営業日。登録時に registered_at＋店舗の営業日境界から算出）, time_low_confidence（時刻低信頼フラグ, bool, オフラインのスキュー補正が信用できない場合 true）, served_at（提供日時）, cancelled_at（取消日時）, cancelled_by（取消者）, cancel_reason（取消理由, ORDER_MISTAKE/QUALITY/DELAY/WRONG_SERVE/SOLD_OUT/CUSTOMER/OTHER）, cancel_chargeable（課金対象か, bool）, was_cooked（bool＝廃棄ロス判定）, remake_of_line_id（作り直し元明細, self, nullable）
order_line_option（明細オプション）	S	明細に付いたオプションのスナップショット	id, order_line_id, option_name_snap（オプション名スナップショット）, price_delta_snap_jpy（追加料金スナップショット, 円）
kitchen_ticket（キッチン伝票）	M	KDS 表示・提供管理の単位	id, order_id, store_id, status（調理状態, NEW/IN_PROGRESS/DONE）, printed_at（印刷日時）, updated_at

2.5 会計・決済

エンティティ	区分	目的	主な属性（代表）
check（会計＝チェック）	M	1卓の精算単位。別会計で複数可	id, table_session_id, store_id, seq_in_session（セッション内連番）, status（会計状態, OPEN/FINALIZED/VOIDED）, subtotal_jpy（小計, 円）, discount_total_jpy（割引合計, 円）, tax_total_jpy（税合計, 円）, total_jpy（合計, 円）, split_type（分割方式, NONE/BY_HEAD/BY_ITEM）, split_count（分割数）, finalized_at（確定日時）, finalized_by（確定者）, business_date（営業日）
check_line（会計明細）	M	どの注文明細をこの会計に割り当てたか（別会計対応）	id, check_id, order_line_id, amount_jpy（金額, スナップショット, 円）, quantity（数量）
check_discount（割引・クーポン）	M	値引き・クーポン・端数調整	id, check_id, type（割引種別, AMOUNT/RATE/COUPON/ROUNDING）, value（割引値, 金額 or 率）, amount_jpy（割引額, 円）, reason（理由）, applied_by（適用者）, applied_at（適用日時）

check_tax_line (税区分別内訳)	M	適格請求書の税率別対価・消費税額	id, check_id, tax_category (税区分, STANDARD_10/REDUCED_8), taxable_amount_jpy (対象額, 円), tax_amount_jpy (消費税額, 円)
payment (決済明細)	M	混合支払い対応 (1会計に複数)	id, check_id, method_type (決済手段, CASH/PAYPAY/CREDIT_CARD/RAKUTEN_PAY), amount_jpy (決済額, 円), status (決済状態, SUCCESS/FAILED/PENDING), external_txn_id (外部取引ID, nullable), tendered_jpy (預り金) / change_jpy (釣り銭 (現金), is_manual_entry (bool=楽天ペイ等), processed_at (処理日時), processed_by (処理者)
refund (返金イベント)	M	会計確定後の訂正。元会計は保持	id, check_id, payment_id (nullable), amount_jpy (返金額, 円), reason (返金理由区分), reason_note (理由備考), executed_by (実行者), executed_at (実行日時), approved_by (承認者, nullable)
receipt (レシート／領収書)	M	発行記録。フェーズ1はPDF	id, check_id, type (帳票種別, RECEIPT/INVOICE), addressee (宛名), proviso (但し書き), issued_at (発行日時), pdf_ref (PDF参照), invoice_reg_no_snap (登録番号スナップショット), tax_lines_snap (税区分別内訳スナップショット, JSON)

2.6 日次締め・売上日報

エンティティ	区分	目的	主な属性 (代表)
daily_close (日次締め)	M	営業日の売上確定・レジ点検	id, store_id, business_date (営業日), status (締め状態, OPEN/CLOSED), closed_by (実行者), closed_at (実行日時), cash_counted_jpy (現金実査額, 円), cash_theoretical_jpy (現金理論額, 円), cash_diff_jpy (過不足, 円)
sales_daily_report (売上日報)	M	締めにより自動生成	id, daily_close_id, store_id, business_date (営業日), sales_total_jpy (売上合計, 円), guest_count (客数), group_count (組数), avg_per_guest_jpy (客単価, 円), discount_total_jpy (割引合計, 円), refund_total_jpy (返金合計, 円)
sales_report_by_payment (手段別内訳)	M	日報の明細	id, sales_daily_report_id, method_type (決済手段), amount_jpy (金額, 円), txn_count (取引件数)
sales_report_by_tax (税区分別内訳)	M	日報の明細	id, sales_daily_report_id, tax_category (税区分), sales_amount_jpy (売上額, 円), tax_amount_jpy (消費税額, 円)
sales_report_by_hour (時間帯別)	M	1時間単位	id, sales_daily_report_id, hour (時刻, 0-23), sales_amount_jpy (売上額, 円), guest_count (客数)

2.7 スタッフ・シフト・勤怠

エンティティ	区分	目的	主な属性 (代表)
staff (スタッフ)	M	人事情報。user と 0.1:1	id, store_id, user_id (nullable), name, role (役割), hourly_wage_jpy (時給, 円), contact (連絡先), is_active (有効フラグ)
staff_availability (勤務可能時間)	M	曜日・時間帯	id, staff_id, weekday (曜日), start_time (開始時刻), end_time (終了時刻)
shift_request (シフト希望)	M	スタッフの希望提出	id, staff_id, target_date (対象日), type (希望種別, AVAILABLE/UNAVAILABLE), start_time (開始時刻), end_time (終了時刻), comment, submitted_at (提出日時)
shift_schedule (シフト表)	M	週単位。作成→公開	id, store_id, period_start (期間開始), period_end (期間終了), status (シフト表状態, DRAFT/PUBLISHED), published_at (公開日時), created_by
shift_assignment (シフト割当)	M	1名の1日の勤務枠	id, shift_schedule_id, staff_id, work_date (勤務日), start_time (開始時刻), end_time (終了時刻), position (担当ポジション)
time_clock (打刻)	M	出退勤	id, staff_id, store_id, work_date (勤務日), clock_in_at (出勤打刻), clock_out_at (退勤打刻), is_corrected (打刻修正済み), corrected_by (修正者), correction_note (修正理由)

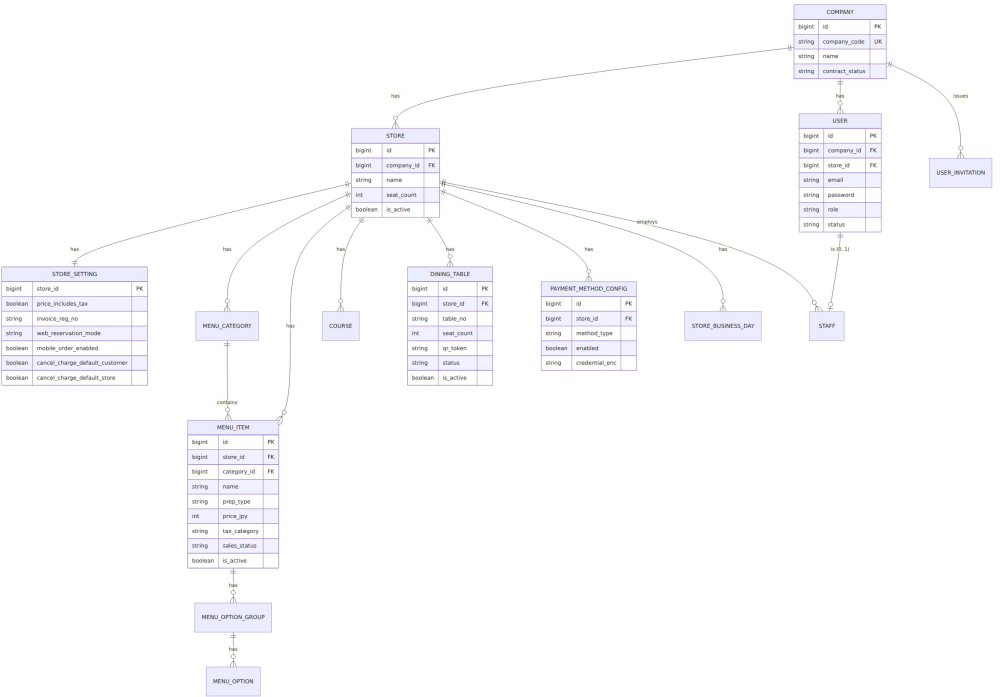
2.8 監査・イベント・通知 (基盤)

エンティティ	区分	目的	主な属性 (代表)
audit_log (監査ログ)	M	重要操作の追記専用証跡	id, company_code (テナント識別子), store_id, actor (操作者, user_id or SYSTEM), action (操作種別, LOGIN/PERMISSION_CHANGE/PAYMENT_SETTING_CHANGE/MENU_PRICE_CHANGE/ORDER_LINE_CANCEL_AFTER_SERVE/CHECK_FINALIZE/CHECK_VOID/REFUI target_type (対象種別), target_id (対象ID), before_summary (変更前サマリ), after_summary (変更後サマリ), ip, device, occurred_at (発生日時)
domain_event (ドメインイベント)	(基盤)	状態変化の時刻付き記録 (分析・AI後付け用)	id, company_code (テナント識別子), store_id, aggregate_type (集約種別, TABLE_SESSION/ORDER_LINE/CHECK/PAYMENT/DAILY_CLOSE), aggregate_id (集約ID, TABLE_OPENED/TABLE_CLOSED/LINE_ADDED/LINE_SERVED/LINE_CANCELLED/MOBILE_ORDER_SUBMITTED/MOBILE_ORDER_ACCEPTED/MOBILE_ORDER_REJIード, JSON), occurred_at (発生日時), actor (操作者)
outbound_message (送信メッセージ)	S	メール／SMSの送信キューと結果 (予約通知・シフト公開通知)	id, store_id, type (送信種別, EMAIL/SMS), to (宛先), template (テンプレート), status (送信状態, QUEUED/SENT/FAILED), sent_at (送信日時), error (

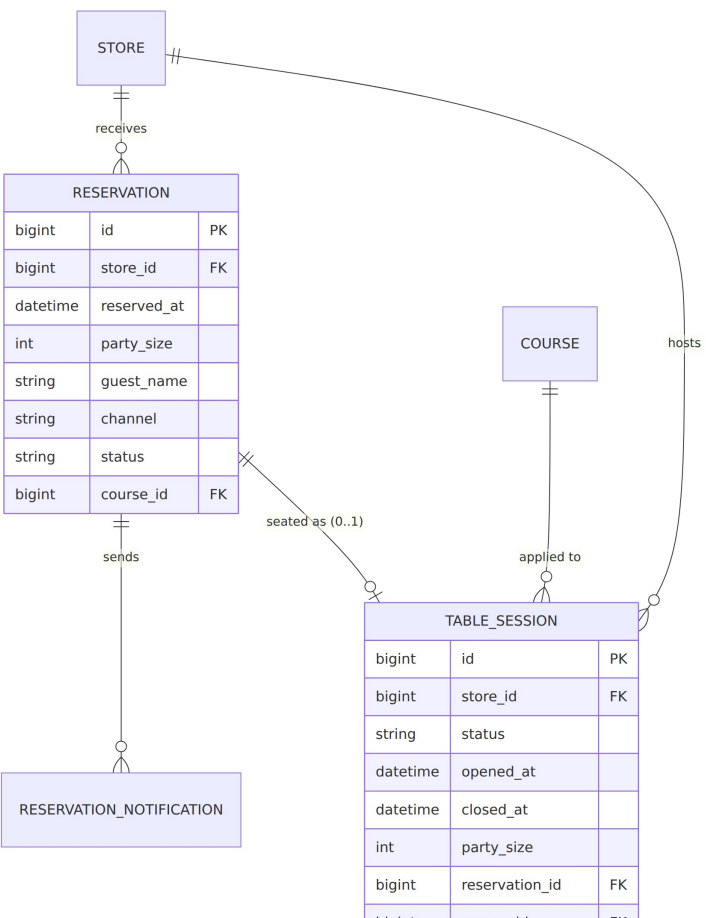
3. ER図 (Mermaid)

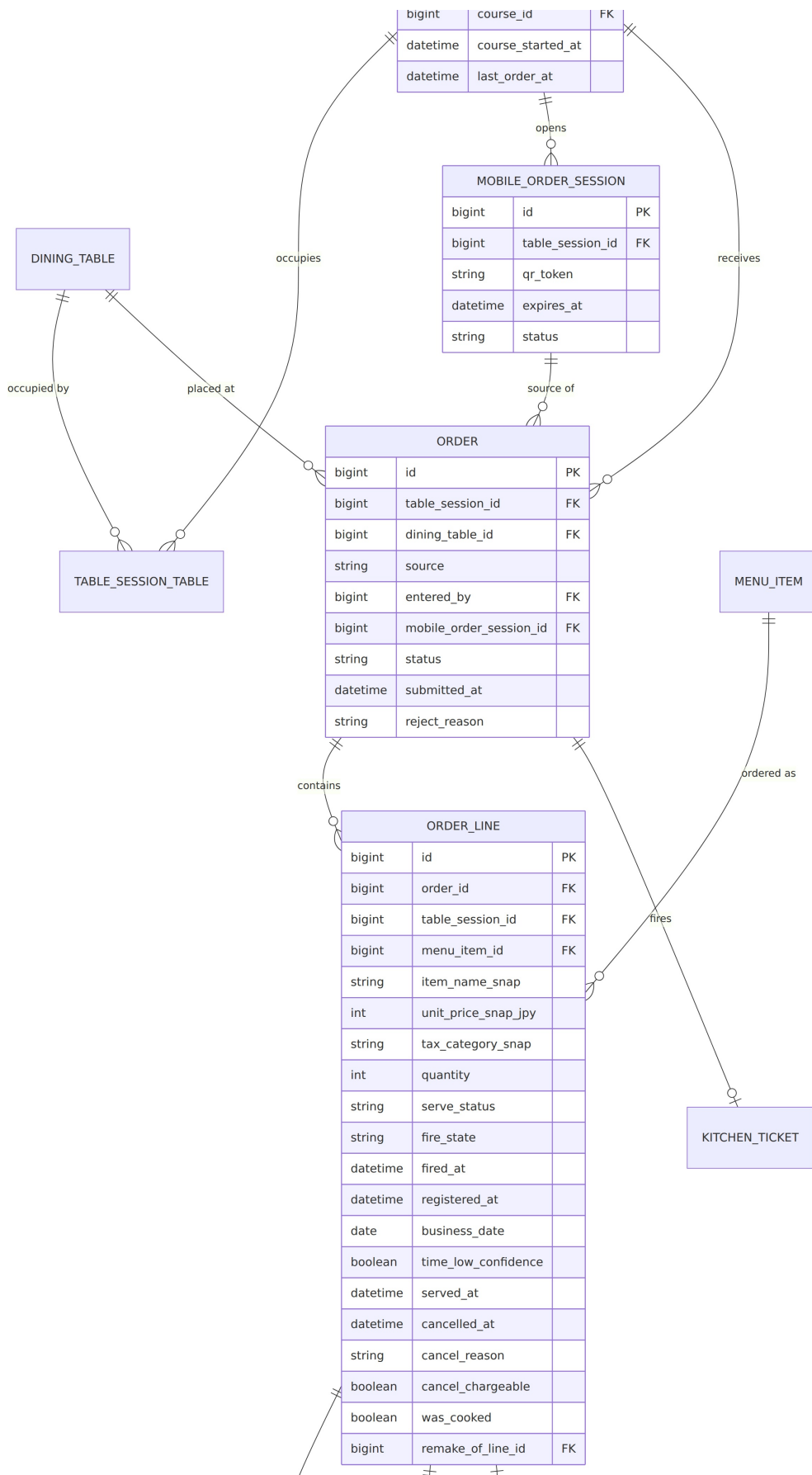
読みやすさのため4つに分割する。属性は代表的なものだけを記載（完全な列定義は 04）。すべての業務エンティティは論理的に company_code と store_id を保持する（図では省略あり）。

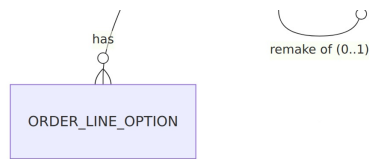
3.1 組織・ユーザー・マスタ



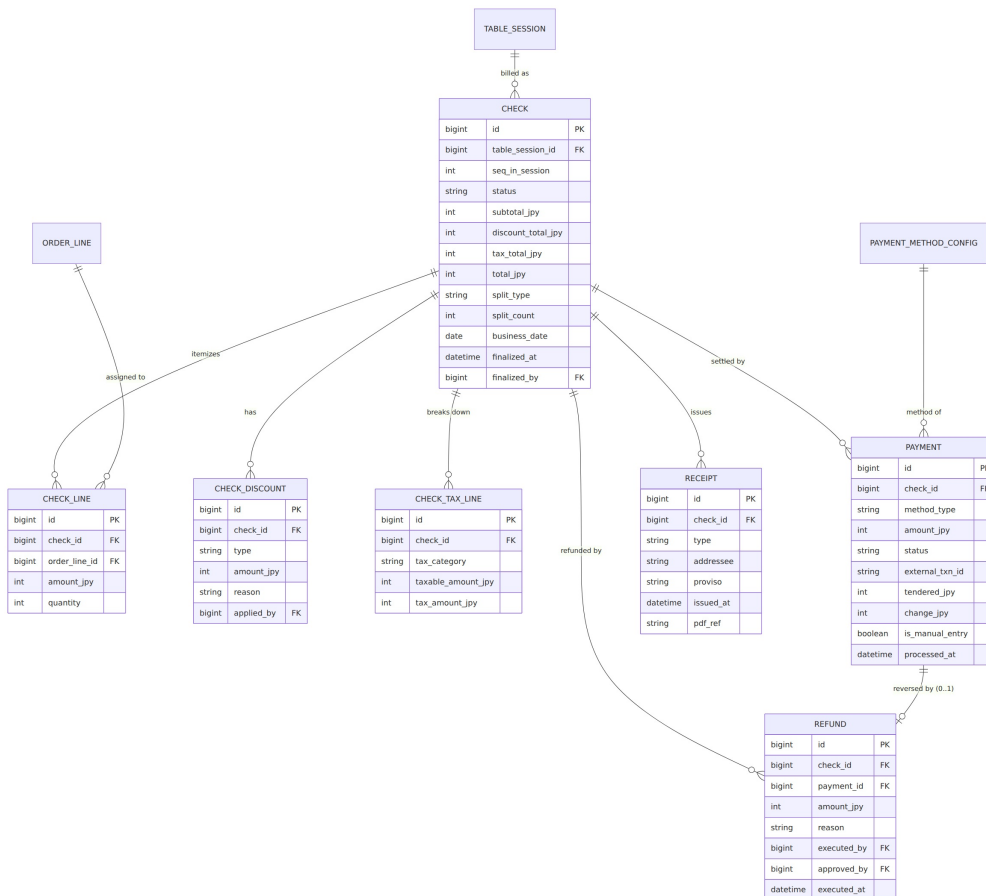
3.2 予約 → 来店 → 注文



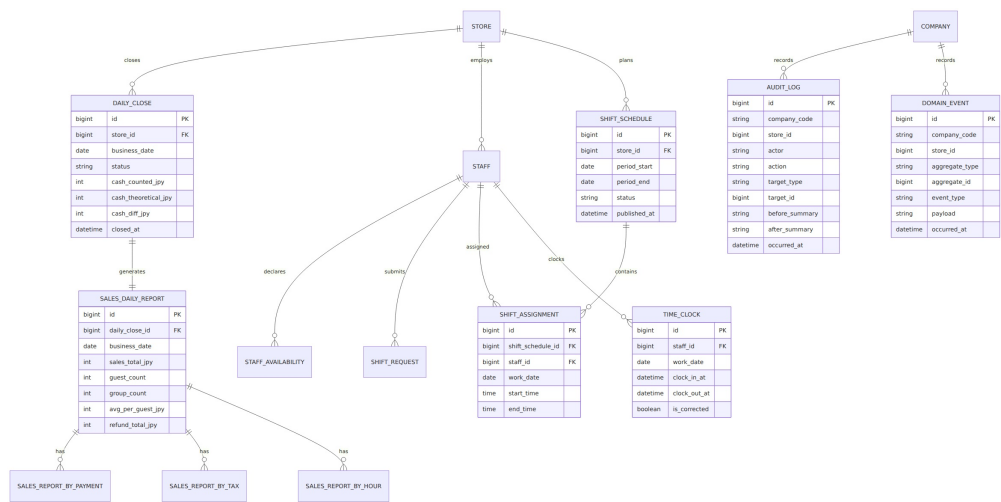




3.3 会計・決済・返金

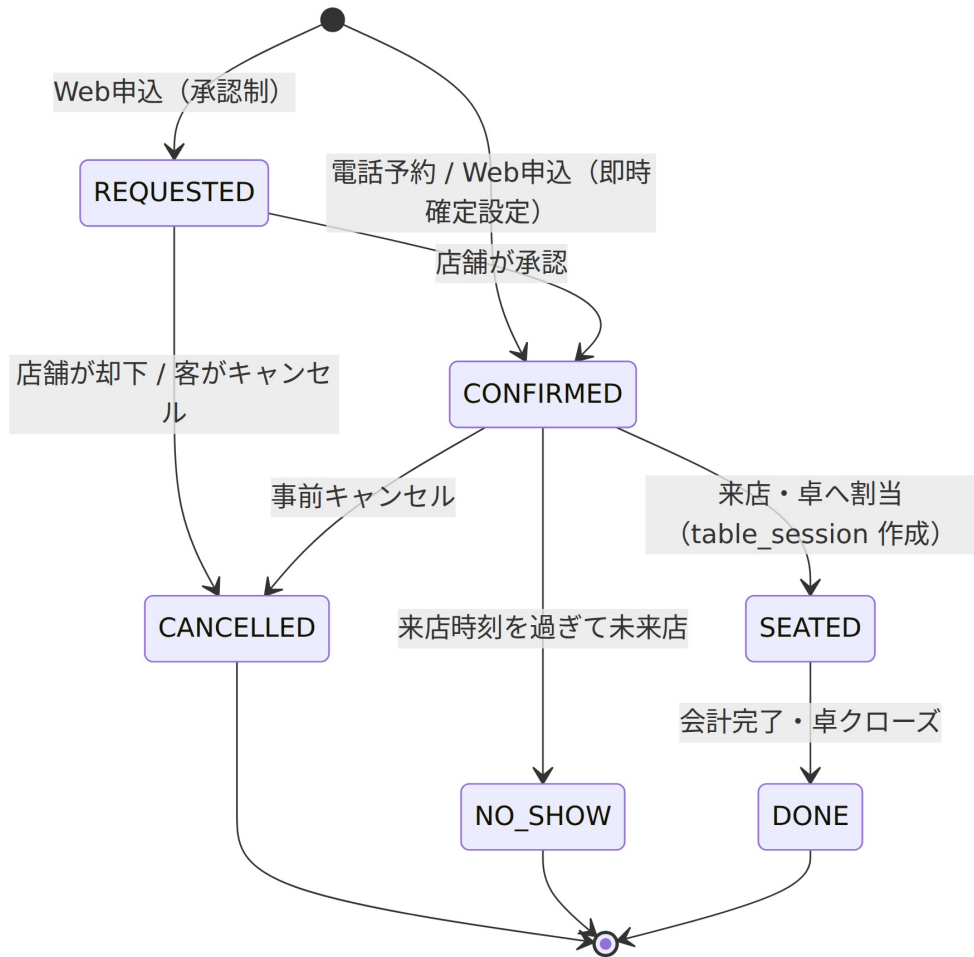


3.4 日次締め・売上日報・勤怠・監査



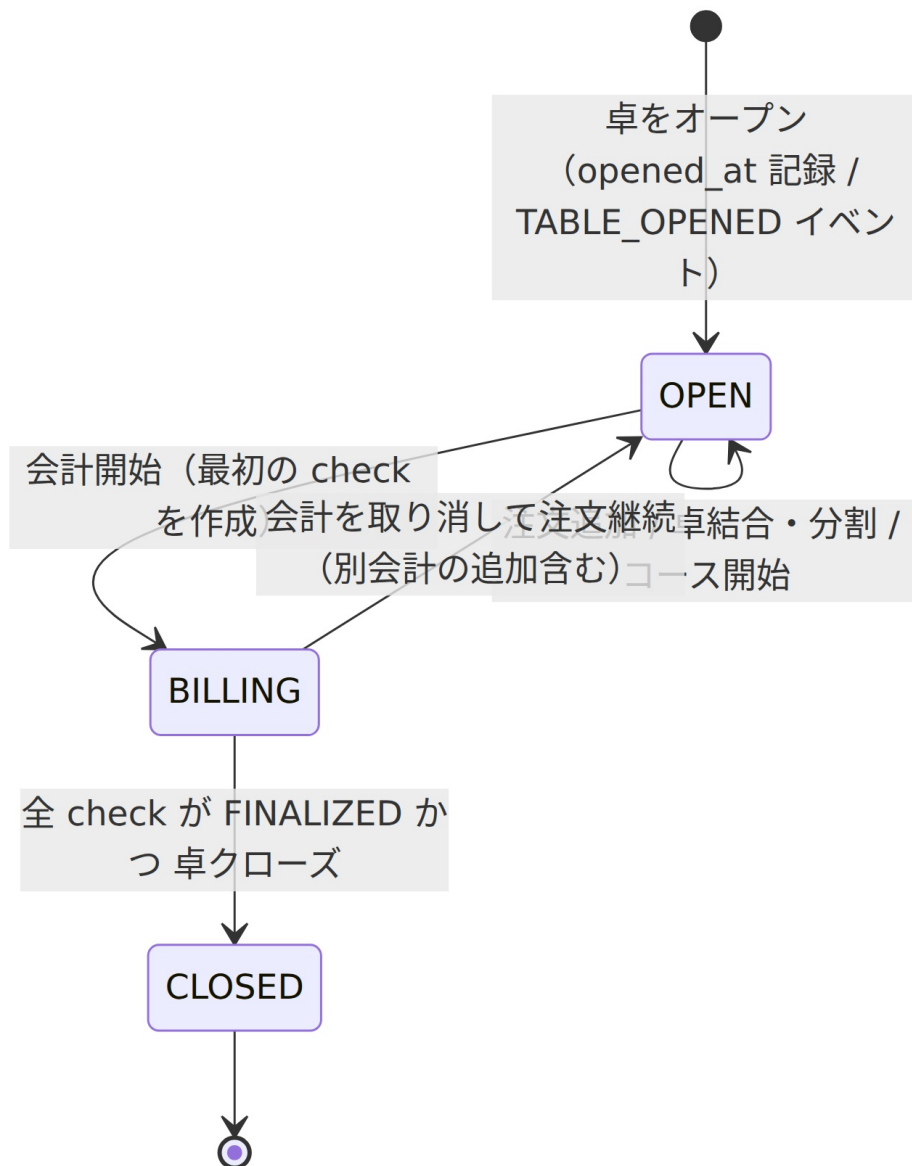
4. 主要な状態遷移 (Mermaid)

4.1 予約 reservation.status



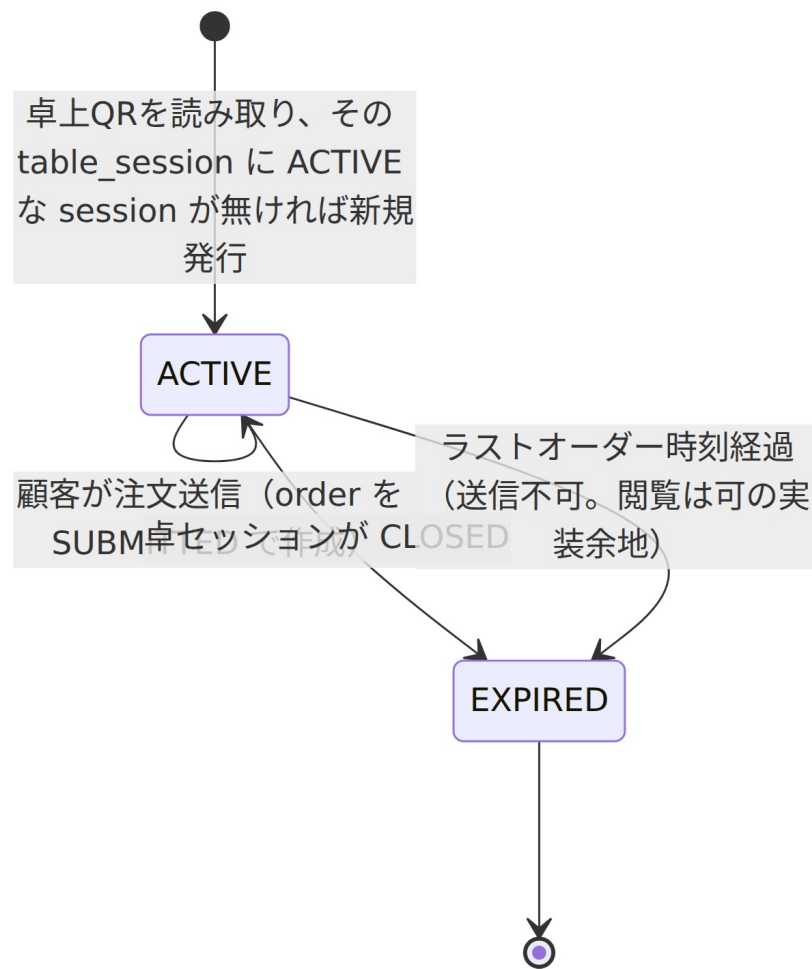
- web_reservation_mode = INSTANT の場合、Web申込は CONFIRMED で開始 (FR-C06 / FR-B09)。
- NO_SHOW と CANCELLED は監査・分析のため区別して保持 (FR-C09)。

4.2 卓セッション table_session.status



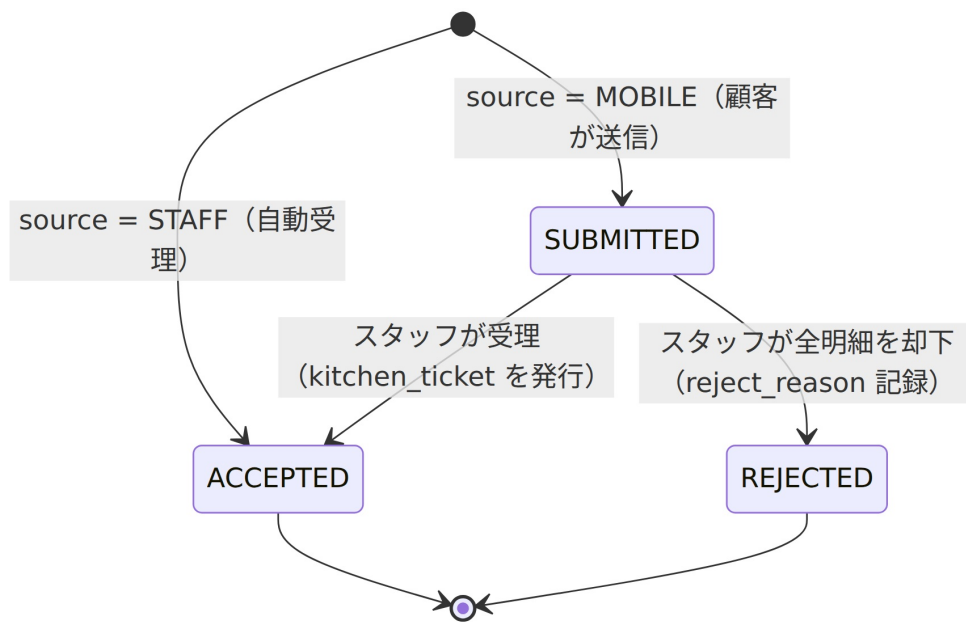
- CLOSED で、紐づく `mobile_order_session` は EXPIRED に連動 (FR-F02)。
- CLOSED 後はこのセッションへの新規注文・会計を禁止 (domain_event: TABLE_CLOSED)。
- 卓の `dining_table.status` (EMPTY/OCCUPIED/BILLING) は、この状態から導出して一覧表示に用いる (FR-E01)。

4.3 モバイルオーダーセッション `mobile_order_session.status`



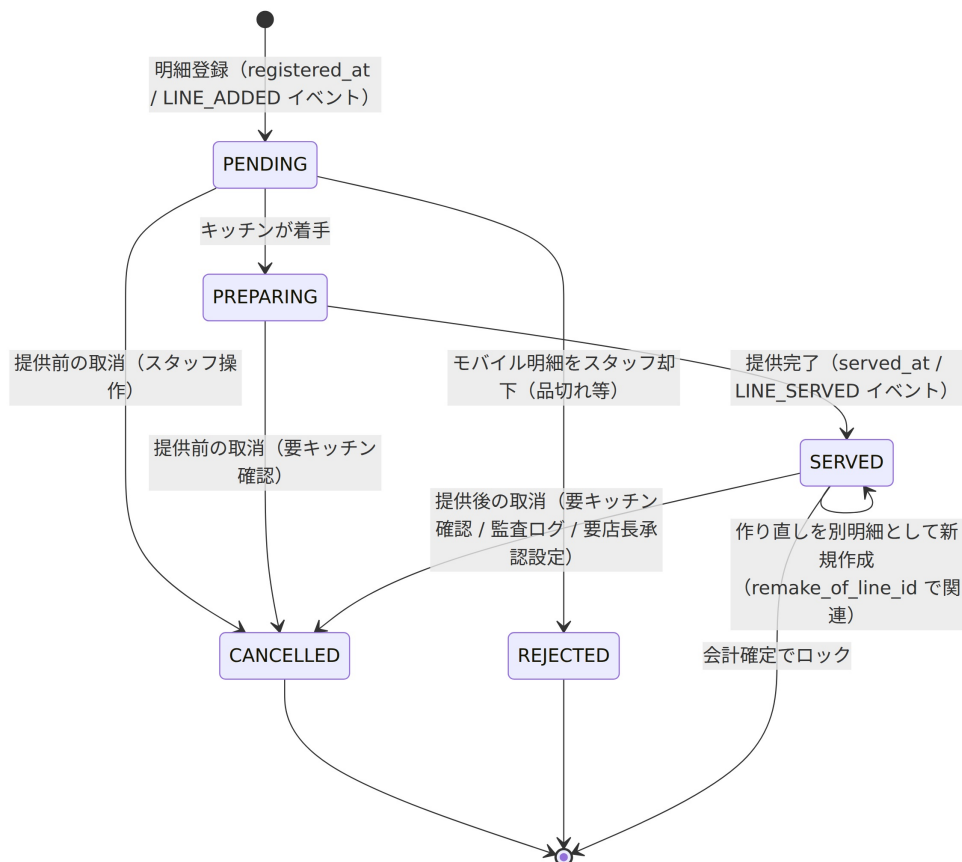
- 同一卓（同一 table_session）に対して同時にACTIVEな mobile_order_session は常に1件とする。QR読み取り時、対象 table_session_id に既にACTIVEな行があればそれを返し（新規行は作らない）、無ければ新規発行する。複数人が同じ卓で別々のスマホから読み取っても、全員が同じ mobile_order_session_id を共有し、同じ注文（カート）状態を見る形になる（客ごとの個別カート分離は行わない）。
- 卓を結合している場合も、結合先のどちらの物理卓のQRを読んでも同じ table_session_id に解決されるため、結合された卓全体で1件の mobile_order_session を共有する。

4.4 注文 order.status（モバイル受理フロー）



- dining_table_id は注文送信時点の物理卓（結合中は結合先のどの卓で入力／QR読取したか）を保持するスナップショット。卓の結合・分割時に、どの order（および配下の order_line）を分割後のどちらの table_session へ移送すべきかを機械的に判定するためのキーとして使う（7章・未決事項2）。
- 一部の明細のみ却下する場合は order は ACCEPTED のまま、該当 order_line を REJECTED にする（4.5）。
- STAFF 注文でも、キッチン未連携・調理前なら明細取消は可能（4.5）。

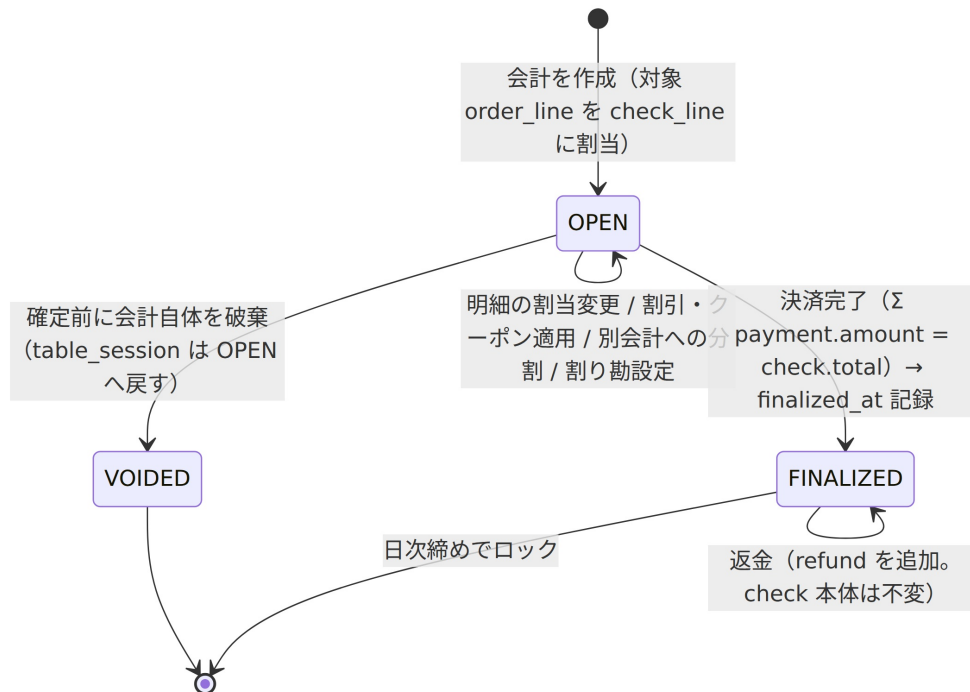
4.5 注文明細 order_line.serve_status



- 取消時は cancel_reason (ORDER_MISTAKE / QUALITY / DELAY / WRONG_SERVE / SOLD_OUT / CUSTOMER / OTHER)、was_cooked (廃棄ロス判定)、cancel_chargeable (store_setting の既定 → スタッフ上書き) を記録 (FR-E03 / FR-E03b / FR-B08)。

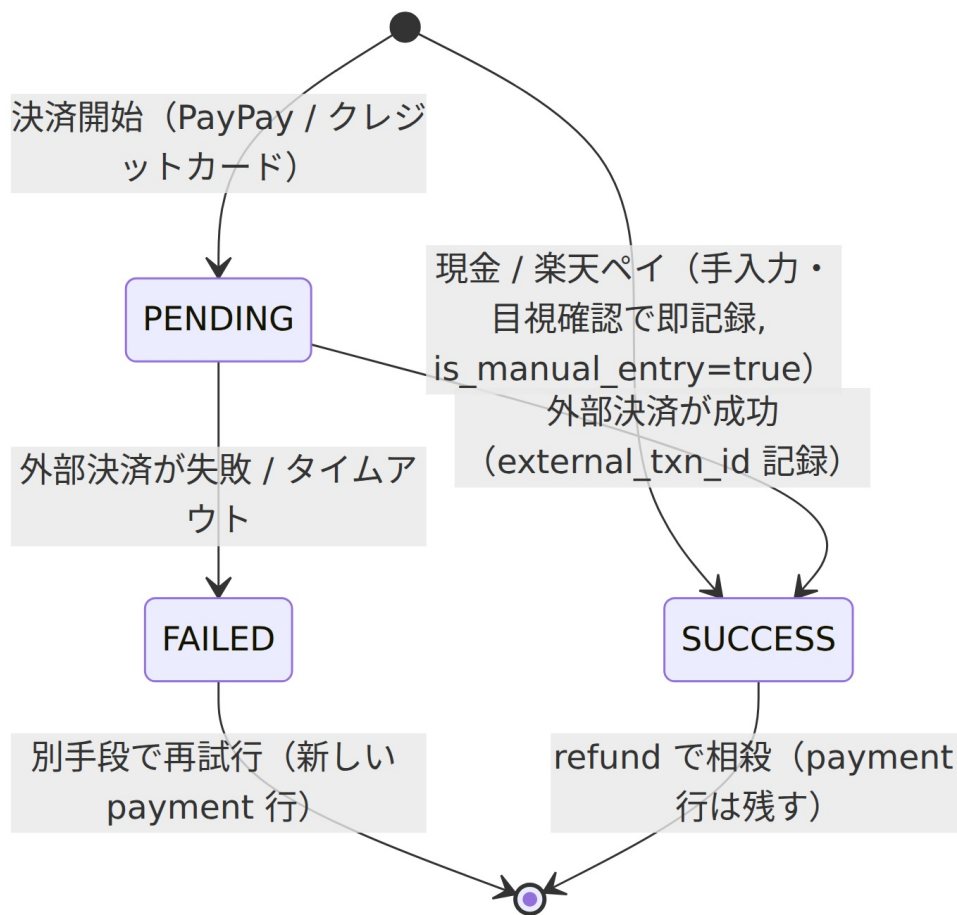
- 「作り直し」は元明細を CANCELLED（理由 QUALITY 等）にしたうえで新規明細を作成し、remake_of_line_id で結ぶ（FR-E03c）。
- SERVED からの取消は domain_event: LINE_CANCELLED と audit_log の両方に記録。
- fire_state（HELD / FIRED）は serve_status と直交する軸で、「調理をいつ始めるか（後出し）」を表す。既定は FIRED。HELD の明細は serve_status = PENDING のまま調理 KDS に出さず、ホールの fire 操作で FIRED に遷移して初めて調理キューへ入る（詳細と KDS 挙動は 04 § 9。段階＝コースステップ構造は持たない簡易版）。

4.6 会計 check.status



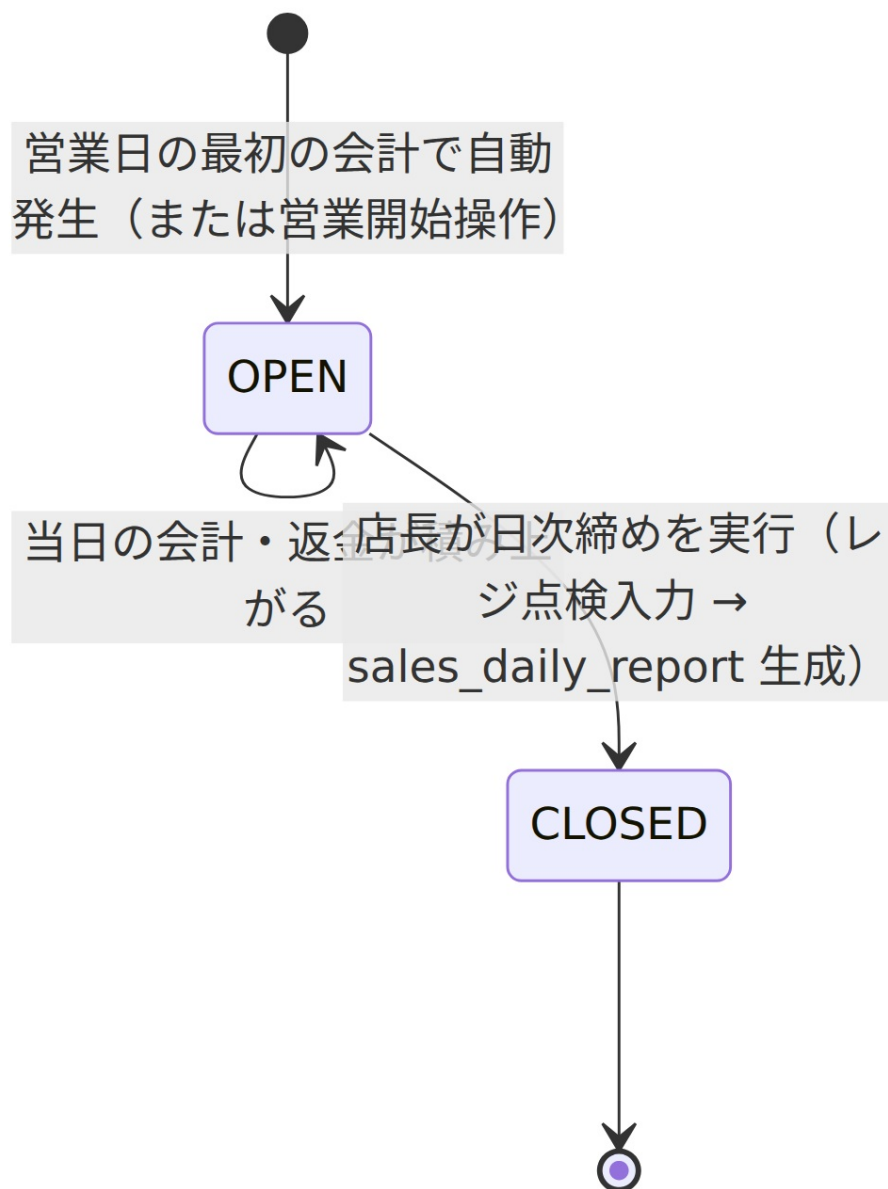
不変条件 (invariant) - $check.total_jpy = subtotal_jpy - discount_total_jpy + tax_total_jpy - FINALIZED$ の必要十分条件: $\sum payment(status=SUCCESS).amount_jpy = check.total_jpy$ (混合支払い可) - FINALIZED 後の金額・明細は不変。訂正は refund (会計確定後) または翌営業日の補正 (日次締め後)。- 1つの order_line は、同一 table_session 内でちょうど1つの check に割り当てられる (別会計で重複割当しない)。

4.7 決済 payment.status



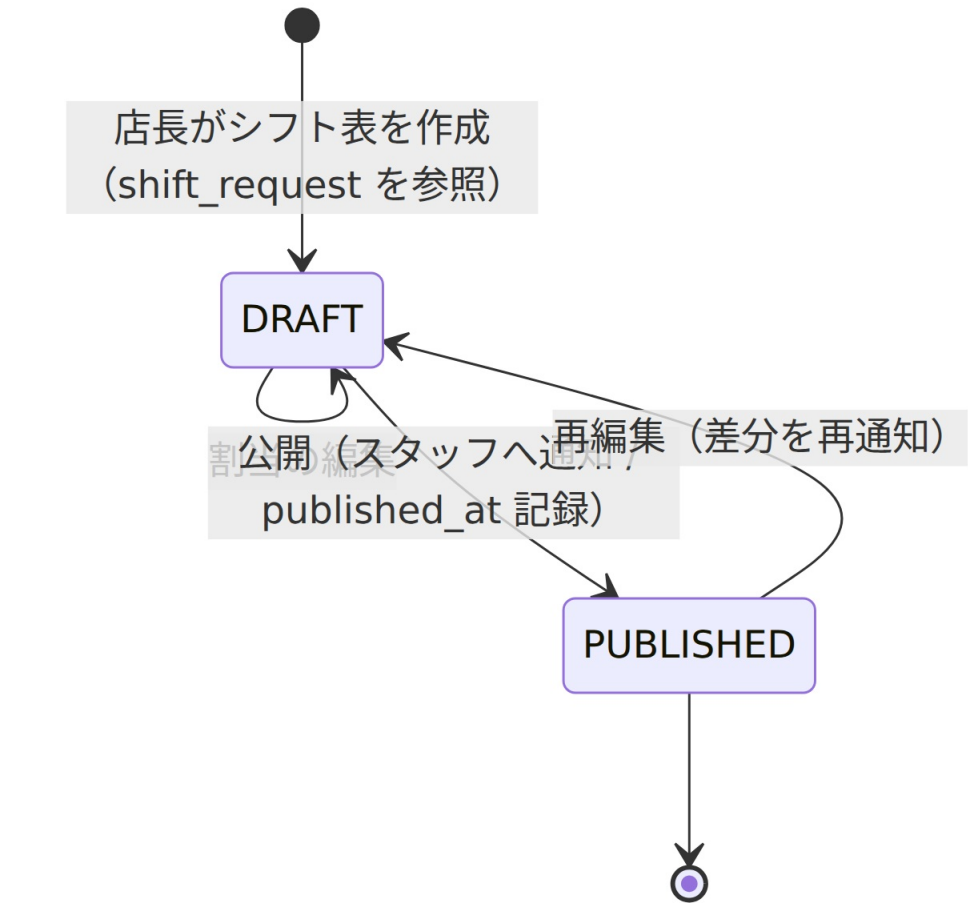
- PayPay 連携失敗時は 'FAILED' を記録し、手入力 (is_manual_entry=true) の 'SUCCESS' で消し込むことを許容 (FR-G06)。
- 楽天ペイ (静的QR/ストアスキャン) は常に is_manual_entry=true (FR-G07b)。

4.8 日次締め daily_close.status



- **CLOSED** 後、その **business_date** の会計・注文明細は変更不可（FR-H01）。以降の訂正は翌営業日の補正として新規計上。
- 締め時に **sales_daily_report** と 3 つの内訳（手段別・税区分別・時間帯別）を生成（FR-H03）。

4.9 シフト表 **shift_schedule.status**



5. 主要なビジネスルール・制約（横断）

#	ルール	関連要件
1	すべての読み書きは company_code でテナントを絞り込む。store_id でさらに店舗を絞る。	FR-A06、NFR-09
2	会計画面に出せる決済手段は payment_method_config.enabled = true のものだけ。	FR-G04
3	menu_item.sales_status が SOLD_OUT / SUSPENDED、または時間帯・期間外の品は、スタッフ注文・モバイル注文とも登録不可。	FR-D03、FR-F03
4	table_session が CLOSED、または last_order_at 経過後は、モバイルオーダー送信不可。	FR-F02、FR-F07
5	モバイル注文の order は ACCEPTED になるまでキッチン連携しない。	FR-F05、FR-F06
6	order_line の SERVED からの取消、check の VOID、refund、discount、daily_close、payment_method_config 変更、menu_item.price 変更、権限変更、データエクスポートは audit_log に必須記録。	FR-J01
7	table_session / order_line / check / payment / daily_close の状態変化は domain_event に記録（分析・AI の後付け）。	FR-K02
8	注文明細・会計明細は作成時点のメニュー名・単価・税区分をスナップショットし、以後マスタ変更の影響を受けない。	FR-K01
9	daily_close.status = CLOSED の営業日のデータは不変。	FR-H01
10	refund は必ず既存の FINALIZED な check に紐づき、元 check / payment は削除しない。	FR-G10
11	オフライン時に作成された order / order_line は復旧後に同期。競合解決規則は 04 で定義（会計・決済はオフライン不可のため対象外）。	NFR-05、NFR-06
12	取消・キャンセルの課金要否は store_setting（客都合 / 店都合の既定）に従い、order_line.cancel_chargeable に確定値を保持。会計時にスタッフが上書き可。	FR-B08、FR-E03

6. 命名・型の指針（04 で最終化）

- テーブル名・カラム名：snake_case、単数形テーブル名。列は英語。
- 主キー：id（bigint / IDENTITY、既存踏襲）。外部キーは <entity>_id。
- 金額：整数、単位サフィックス _jpy。時刻：timestampz（列名 _at）。日付：date（business_date 等）。
- 列挙（enum）：文字列で保持し、アプリ側で定数管理（DB enum 型は使わない）。本書の各状態値を初期セットとする。

- テナント/店舗別: `company_code` (`varchar(20)`、既存準拠) と `store_id` を全業務テーブルに付与。
- 監査列: `created_at` / `updated_at` は全テーブル、`created_by` / `updated_by` は業務テーブル。

7. 未決事項（04 または実装スパイクへ）

1. `company` と既存 `users` (← `company_code` 文字列キー) の物理的な結合方法: `users` に `company_id` FK を足すか、`company_code` 参照のままにするか。解決済み (04.architecture.md § 3.1・§ 4.3) : `company_id` (サロゲートキー) を主キーとし、他の全FK (`store_id` → `store_id` 等) と一貫させて `store・users・user_invitation` に実FKの `company_id` BIGINT REFERENCES `company(id)` を追加する。`company` 自身が `company_code` (UK) を持つため、`store・users・user_invitation` には `company_code` 列は一切持たせない (`users.company_code` も削除し、複数ユニーク制約は `company_code + email` から `company_id + email` に変更する)。登録・ログイン画面で利用者が入力する `company_code` は、`company` テーブルへの参照検索 (`WHERE company_code = ?`) で `company_id` に変換してから使う。それ以外の業務テーブルの `company_code` 列は、`store_id` から `store.company_id` を辿れば会社を特定できるため実FKにはせず、テナント絞り込み用の非正規化コピー (FK制約なし) のまま維持する。
2. 卓の「結合・分割」の詳細 (FR-E06 は \$) 。
 - 分割時にどの `order` ≠ `order_line` を移送すべきかの判定キーが無い問題 一部解決 : `order` に `dining_table_id` (注文時点の物理卓のスナップショット) を追加。結合中の来店を分割する際は、分割後の各卓に対応する `dining_table_id` を持つ `order` (およびその配下の `order_line`) を新しい `table_session` へ付け替える、という機械的な移送ロジックが組める (4.4節参照)。
 - フェーズ1方針決定: 会計開始後 (`table_session.status = BILLING`、すでに `check` が作成された後) の卓分割はフェーズ1では対応しない。分割が必要な場合は、対象の `check` を VOID して `BILLING` → `OPEN` に戻し (4.6)、`OPEN` 状態で卓を分割してから会計をやり直す運用とする。`check_line` で紐づけ済みの `order_line` の付け替え、`check.total_jpy` / `check_tax_line` の再計算、適用済み `check_discount` の再配分、`payment` 済み・FINALIZED 済みケースの返金経路までは設計が広く波及するため、フェーズ2以降で対応する。
3. オフライン同期の競合解決規則 (後勝ち/マージ/要確認) とクライアント側の一時的ID探索。
 - 解決済み (04.architecture.md § 1決定表3・§ 9) : 競合解決規則は実装しない。オフライン継続を「注文明細の新規追加のみ」に限定し、数量変更・取消・会計・決済はオンライン必須とすることで、更新競合そのものを設計上発生させる余地をなくす (追加どうしは独立しているためマージも不要)。
 - 解決済み (同 § 9・§ 4.6) : クライアント一時IDの形式は「端末ID (サーバがデバイス登録時に発番、`store` 短縮コード+店内連番、例 S12-07) + 端末内でグローバルに単調増加する連番」の合成文字列 (例 S12-07-000123) とし、`customer_order・order_line・order_line_option` の各レコードの `client_ref_id` (`VARCHAR(40)`) に保持する。各テーブルの `UNIQUE(..., client_ref_id)` と `staff_device.last_accepted_seq` でサブツリー全階層の冪等性を担保し (親だけでなく明細・オプション単位で再送を無視)、端末の再セットアップ時は端末IDを再発番してIDは再利用しない。
 - 解決済み (同 § 9・§ 1決定表3) : 同期ペイロードはフラットなレコード配列+FK列に一時IDの形式。作り直し参照は `remake_of_line_id` (同期済み明細を指す確定ID) と `remake_of_line_ref` (同一バッチ内の未同期明細を指す一時ID。ペイロード専用でテーブル列にはしない) に分けて持つ。バッチ内の依存順はアプリ層のトポロジカルソートで解決する (DB の DEFERRABLE 制約は採用しない)。
 - 解決済み (同 § 9) : 同期レスポンスは、エンベロープ (`server_received_at` / `last_accepted_seq`) + 送信レコード全件に対応するフラットな `records[]` を返す。各要素は `type` / `client_ref_id` / `id` / `status` (`INSERTED` / `DUPLICATE` / `REJECTED`) / 任意の `server_fields`・`error`。再送しても各 `client_ref_id` に同じ `id` を返す。
 - 解決済み (同 § 9) : クライアント側の一時的ID+確定ID対応表は、対象レコードが属する `table_session` が `CLOSED` になるまでローカル保持し、以後そのセッション分を破棄する (`CLOSED` を長期間観測できない場合の固定TTLをバックストップに置く)。サーバ側は `client_ref_id` 列を \$10 の保持期間まで永続保持するため、これはクライアント端末のローカルコピーの寿命のみを定める。
 - 解決済み (同 § 9) : 部分失敗はハイブリッドで扱う。サブツリー前提の検証 (親 `customer_order`・対象 `table_session` の受付可否・構造整合性) は all-or-nothing で、落ちたらごとくロールバックし全配下を `SKIPPED`。個々の `order_line` / `order_line_option` の業務検証は部分コミットで、落ちた明細は `REJECTED + error.code` (`SOLD_OUT` 等の初期セット)、その配下オプションは `SKIPPED` で返す。リトライ可否は明示フィールドを置かず、`error.code` 付き `REJECTED` = 永続的失敗 (自動再送せず スタッフへエスкаレーション)、一時的失敗は 5xx / タイムアウト等のトランスポート層で判定してバッチ全体を指数バックオフ再送、と切り分ける。残り (却下=提供済み料理の監査/廃棄ロス連携、HTTP ステータス規約) は実装スパイクで確定。
 - 解決済み (04.architecture.md § 9・§ 1決定表3) : 遅延同期が `CLOSED` の `table_session` / `FINALIZED` の `check` に着地した場合、フェーズ1では同期APIが `SESSION_CLOSED` で `REJECTED` を返し、拒否された明細は業務的に回収しない。`check` への明細追加・補助会計・refund 等の金銭リカバリは行わず、回収不能=廃棄ロス/サービス提供分として `domain_event` (監査) に記録するのみとする。売上計上・追加請求はせず、代金回収経路 (追加請求・翌営業日補正) の整備はフェーズ2以降とする。
 - 解決済み (04.architecture.md § 9・§ 1決定表3) : オフライン中に許可するのは新規レコードの作成のみ。すでにサーバへ永続化済みの行 (オンラインで作成された `order_line` 等、`client_ref_id` を持たない行) への更新は、`serve_status` の `PENDING` → `SERVED` 変更を含めオフラインでは一切不可とする。`updated_at` / 専用 `version` 列を初期表示時にクライアント保持し復帰時にサーバ現在値と突き合わせる素観的ロックでの解禁は採用しない (長時間オフラインでは版の不一致が高頻度で発生し、エラー後の手動照合がかえて増えるため)。こうした更新はオンライン復帰後に行う (アプリがオフライン時に当該操作を抑止するか、意図をローカルキューへ退避して復帰後に通常のオンライン更新として再生する)。なお、オフラインで新規作成された明細が初回同期時に自前の `serve_status` / `served_at` を初期値として持ち込むことは認める (下記「`kitchen_ticket` 抑止」=案1で確定)。これは「INSERT 時の初期状態」であって既存行の更新ではないため、上記の禁止対象には当たらない。
 - 解決済み (04.architecture.md § 9・§ 1決定表3) : スナップショットの鮮度は「オフライン中は端末が保持するメニューマスタで確定」とする。オフライン作成明細の `item_name_snap` / `unit_price_snap_jpy` / `tax_category_snap` (およびオプションの `option_name_snap` / `price_delta_snap_jpy`) は、注文時点で端末ローカルのメニューキャッシュから採ったスナップショット値をそのまま採用する。オンライン復帰時、サーバは現行の `menu_item` / オプションマスタで再計算・再価格付けをしない (オフライン中にマスタの価格・名称・税区分が変わっていても、端末が持っていた値で確定させる)。※対象商品がオフライン中に `SOLD_OUT` / `SUSPENDED` / `is_active = false` になっていた場合の受入可否は、下記「`kitchen_ticket` 抑止 (案1) の細目」で解決済み (`serve_status` で分岐: `PENDING` は `REJECTED`、`PREPARING` / `SERVED` は却下せず `INSERT`)。
 - 解決済み (04.architecture.md § 9) : オフライン作成レコードの時刻は「端末時刻+スキュー補正」で確定する。
 - 端末は同期バッチ送信時に `client_sent_at` (送信時点の端末時計値) を含める。サーバは `offset = server_received_at - client_sent_at` を求め、バッチ内の各オフライン時刻 (`order_line.registered_at`、`customer_order.submitted_at`、対応する `domain_event.occurred_at`、A案を採る場合の `order_line.served_at`) に一律加算する。「オフライン中はオフセット一定」を前提とすることを許容する。
 - 補正後の値は原則そのまま格納し、低信頼フラグを立てる (分析側が除外可能にする)。ただし補正後が時系列的にあり得ない場合 (`table_session` 開始前、`server_received_at` より未来等) は、その時刻だけ `server_received_at` で置換する。低信頼フラグは永続化し (真偽値列。名称・配置は 04 § 4.6 で最終化)、同期レスポンスの `server_fields` でもエコーバックする。
 - `offset` の許容上限 X (これを超えたら定常ドリフトの範囲外とみなす閾値) は運用設定値として持つ (システム全体の既定値。店舗別オーバーライドはフェーズ2以降)。端末のクロックドリフトは機種・OS の特性であり店舗業務には依存しないため `store` 単位では持たない。格納方式 (汎用設定テーブル新設かアプリ設定か) と、既定値・安全に設定できるレンジは実機のドリフト実測に基づき実装スパイクで確定する。
 - 解決済み (04.architecture.md § 9・§ 4.6) : オフライン明細の `business_date` 帰属。
 - `order_line` に `business_date` DATE NOT NULL 列を追加する (A2)。`guest_check.business_date` (= 精算日) とは別に、明細ごとに「注文された営業日」を持つ。
 - 導出は補正後 `registered_at` + 店舗の営業日境界 (`store_business_day`) から算出する (B1)。
 - ただし `time_low_confidence = true` の明細は `registered_at` を信用せず、`business_date` を紐づく `guest_check.business_date` で決める (B2 フォールバック。低信頼フラグと連動)。
 - 解決済み (04.architecture.md § 9) : B1 の算出先が既に `daily_close = CLOSED` の営業日だった場合 (端末が締めをまたいでオフラインだったとき) は D2 を採用する。
 - 当該明細は拒否せず受け入れ、`business_date` を月曜等の締め済み日ではなく現在のオープン中の営業日に設定して当日計上する。補正後 `registered_at` は実際の注文時刻として列にそのまま残す。
 - 明細には「前営業日からの遅延計上」を示す理由コード/フラグを立てる。列名と `sales_daily_report` での前日遅延計上の表示方法 (内数・脚注など) は実装スパイク送り。
 - 締め済みの `daily_close` 側には「前日分の遅延計上あり」の通知・フラグは原則行わない (`CLOSED` の不変性を維持する)。
 - 解決済み (04.architecture.md § 9) : KDS のチケット表示順の基準時刻は `registered_at` (オフラインは補正後) とする。`kitchen_ticket` の生成時刻 (`printed_at` = オフライン分は同期到着時刻) は使わない。`kitchen_ticket` は `customer_order` 単位のため、ソートキーは注文入力時刻 (`customer_order.submitted_at` = 配下 `order_line.registered_at` の最小値)。`time_low_confidence` の明細は表示位置がずれ得るが、KDS は不変データを持たない一時表示のため許容する。どの明細を KDS に出すか (提供済みオフライン分の抑止) は下記「`kitchen_ticket` 抑止」で別途決める。

- 解決済み (04.architecture.md §9) : 遅延同期時の kitchen_ticket 抑止は案1 (バイロードに serve_status を載せる) で確定。
 - オフライン作成の order_line は、同期バイロードに端末上での serve_status (および served_at) を INSERT 時の初期状態として含める。サーバは受信時に serve_status が PENDING 以外 (提供済み等) の明細については、新規 kitchen_ticket を KDS に鳴らさない。
 - serve_status = PENDING のオフライン明細は従来どおり kitchen_ticket を発行し、KDS に表示する (表示順は上記のとおり registered_at 基準)。
 - この仕組みは「オフラインは新規追加のみ・既存行の更新は不可」と矛盾しない (サーバの操作は INSERT 1回で、既存行への UPDATE は発生しないため)。
- 残課題 (フェーズ1スコープ外または実装スバイク送り。凍結の妨げにはならない) :
 - オフライン許容時間の上限 (端末が何時間オフラインのまま新規注文を受け付けてもいか。上限到達時に読み取り専用へ落とすか否か。daily_close の境界——時ID対応表の固定FFLとの整合)——フェーズ1は上限を設けない (端末は無制限にオフライン明細を溜められる) で確定。上限到達時の読み取り専用移行を含む上限設計は、offset 許容上限 X と同じ運用設定の器に載せてフェーズ2以降で行う。スキーマにもサーバの受け入れ分岐にも影響しない (遅延着地は D2 / SESSION_CLOSED の既存分岐で処理される)。
 - time_low_confidence 列の最終的な名称・配置・端末の生時刻を別列に残すか、復帰時に端末時計をサーバ同期するか。解決済み (04.architecture.md §4.6・§9) : 列名は order_line.time_low_confidence (BOOLEAN NOT NULL DEFAULT false) で確定。理由 (X 超過 / 時系列破綻置換) は区別せず真偽値1本。配置は order_line のみ (customer_order には持たせない。submitted_at は配下明細の registered_at の最小値であり、信頼性が要る処理は明細側のフラグを参照する)。スキュー補正前の 端末時計値は order_line.registered_at_device_raw (オフライン作成のみ、オンライン分は NULL) に残す。復帰時に端末時計をサーバへ同期し直す仕組みはフェーズ2以降。
 - kitchen_ticket 抑止 (案1) の細目 :
 - 解決済み : kitchen_ticket は従来どおり customer_order 単位で1件発行する (行単位に分割しない)。オーダー内の全明細が SERVED (提供済み) の場合のみ、そのチケットを status = DONE で作成し KDS には表示しない (レコードは監査・スループット分析用に残す)。1つでも PENDING 明細を含む オーダーは、チケットを通常どおり KDS に出し、調理ビューに表示する明細を serve_status = PENDING のものだけに絞る (SERVED 明細は既に serve_status 済みなので調理ビューから除外。KDS 表示クエリの フィルタで実現し、スキーマ変更は不要) = 方式 G1。
 - 解決済み (04.architecture.md §9) : 「鳴らさない」の強さはフェーズ1は完全非表示で確定。status = DONE のチケットは KDS のどのレーンにも出さず (DB には監査・分析用に残す)、アラート 抑止のみに留めて「ミュートの照合レーン」に出す案は採らない。照合レーン自体がフェーズ1では未実装で、必要になればフェーズ2で照合レーンごと格上げする。
 - 解決済み (04.architecture.md §9) : オフライン中に SOLD_OUT / SUSPENDED / is_active = false になっていた商品の明細は、serve_status で分岐する。PENDING (未提供) は従来どおり REJECTED、PREPARING / SERVED (オフライン中に手作業で厨房へ通して調理中・提供済み) は却下せず INSERT し、売上をスナップショット値で計上したうえで domain_event に監査記録する。
 - 解決済み (04.architecture.md §9・§4.7) : 案1で「オフライン提供済み」明細から同期時に status = DONE で生成した伝票は、KDS の滞留時間・スループット・平均調理時間などの厨房指標から除外する (売上・提供実績カウントには含める)。通常営業で DONE 遷移した G1 の伝票は指標に含める。識別のため kitchen_ticket.offline_settled (BOOLEAN NOT NULL DEFAULT false) を追加し、案1が DONE 伝票を作るときのみ true を立てる。
- 4. domain_event の粒度と保持・アーカイブ方針 (件数が多い。パーティション／コールドストレージ)。解決済み (04.architecture.md §1決定表行4・§8) : 粒度は aggregate_type (TABLE_SESSION / ORDER_LINE / CHECK / PAYMENT / DAILY_CLOSE) ごとの主要な状態変化のみを記録し、カラム単位の差分履歴は対象外 (必要なら audit_log 側で扱う)。書き込みは Transactional Outbox パターン (業務トランザクションと同一トランザクション内で INSERT)。フェーズ1では分析基盤への配信 (Kafka / CDC) は持たずテーブル蓄積のみとし、occurred_at の月次レンジパーティション (pg_partman で自動管理) とする。直近13か月をオンライン参照可能に保持し、それを超えたパーティションは S3 互換ストレージへ JSONL でエクスポートしてデタッチ、04 §10 のデータ保持方針 (10 年) を経過したらコールドストレージ側でも削除する。
- 5. モバイルオーダーの qr_token の設計 : 単固定トークン+セッション毎の短命トークンの2層にするか。失効・再発行の運用。解決済み (04.architecture.md §1決定表行5・§7.4) : 2層構成を採用。dining_table.qr_token は卓に印刷される固定トークンで、店舗設定画面から手動再発行可能 (再発行時は旧トークンを即時無効化)。読み取り時に対応する table_session が OPEN であれば mobile_order_session を新規発行し、セッション固有の短命 qr_token (実体はランダム文字列、有効期限は卓クローズ or 発行から 12時間の早い方) をクライアントへ返す。table_session が CLOSED になると紐づく全 mobile_order_session を EXPIRED する (4.3)。短命トークンは JWT でなくサーバ側セッション参照とし、クライアントの sessionStorage に保持してリクエストヘッダで送る。
- 6. 税計算の丸め (会計単位 / 明細単位)、端数調整 (ROUNDING ディスカウント) の扱い。解決済み (04.architecture.md §1決定表行6・§6.4) : 消費税額は check_tax_line の税区分 (STANDARD_10 / REDUCED_8) ごとに、対象額合計へ税率を乗じたうえで1会計につき1回だけ端数処理する (インボイス制度の「1請求書につき税率ごとに1回」に対応)。明細単位では税額を計算・表示しない (合計金額の安否スレ回避)。端数処理方式は切り捨て (store_setting.tax_rounding の既定値 FLOOR。CEIL / ROUND へ店舗別に変更する余地は列として残すが、フェーズ1のUIでは変更不可＝固定運用)。現金精算等で生じる1円未満の調整は check_discount(type = 'ROUNDING') として明示的に記録する。
- 7. receipt の適格請求書レアウトとPDF生成方式 (サーバ生成 / テンプレート)。解決済み (04.architecture.md §1決定表行7・§6.5) : PDF 生成方式はサーバサイド生成で確定。openhtmltopdf (org.xhtmlrenderer:openhtmltopdf-pdfbox) で Thymeleaf の HTML テンプレートから 直接 PDF バイト列を生成する (外部プロセス起動なし。wkhtmltopdf 等の外部バイナリ依存は避ける。ドキュメントPDF化で用いた wkhtmltopdf 方式は開発ドキュメント用途に限定)。生成 PDF はオブジェクトストレージ (本番ホスティング確定後に選定、開発中はローカル or MinIO で代替) に保存し receipt.pdf_ref にキーを保持する。receipt は発行時点の適格請求書登録番号 (invoice_reg_no_snap) と税区分別内訳 (tax_lines_snap JSONB) をスナップショットで持つ。適格請求書の具休レアウト (記載項目の配置・様式) は、上記テンプレートの実装詳細として実装時に確定する。
- 8. staff と user の一体化度合い (打刻のみの非ログインスタッフを許容するか)。解決済み (04.architecture.md §1決定表行8・§4.9) : 03 の設計 (staff は user と 0..1:1、staff.user_id は nullable) をそのまま確定し、一体化 (staff.user_id を NOT NULL 化、または user への人事列統合) は採らない。
 - 非ログインスタッフの許容 : システムにログインしないスタッフ (入れ替わりの多いホール・キッチン のアルバイト等) は staff.user_id IS NULL の行として登録する。email / パスワード・2要素認証は不要で、名前・時給・役割だけでシフトと打刻の対象にできる。
 - ログイン要否の切り分け : レジ・会計・設定変更などシステム操作を行う役割 (店長、社員) だけ user を作成し staff.user_id で紐付け。勤怠・シフト系 (time_clock / shift_request / shift_assignment / staff_availability) はすべて staff_id を参照するため、非ログインでも打刻・シフトは完結する。
 - 役割 (a) : user.role (enum: OWNER/MANAGER/HALL/KITCHEN/PARTTIME) は権限、staff.role (自由文字列) は勤怠・シフト画面でのポジション表示ラベル、と役割を分けて両方残す。統合しない。
 - 表示名 (b) : 勤怠・シフト画面の表示名は staff.name を正とする。user.name はアカウント表示用。
 - 多重度 (c) : フェーズ1では 1 user = 1 staff (同一店舗) に限定する。1人が複数店舗の staff 行を持つ多店舗兼務はフェーズ2以降。
 - staff を持たない user (d) : 本部の OWNER など勤怠対象でない user は staff 行を作らなくてよい (許容)。逆向き (staff あり user なし) が非ログインスタッフ。
 - 打刻の本人確認 (e) : 非ログインスタッフの打刻UIは「店舗共有端末のスタッフ一覧から選択」のみとする。PIN 等の個人認証 (staff への clock_pin 列追加など) はフェーズ2以降。打刻の修正は従来どおりログインユーザーが行い time_clock.corrected_by に記録する。
 - 監査ログの actor は user_id or SYSTEM のままで、非ログインスタッフは actor に登場しない。
- 9. 売上日報の「客数」「組数」の定義 (table_session.party_size の合計 / table_session 件数)。解決済み (04.architecture.md §1決定表行9・§4.8) : 以下で確定。スキーマ変更は不要 (既存の sales_daily_report.guest_count / group_count / avg_per_guest_jpy で足りる)。
 - 集計母集団 : status = CLOSED かつ FINALIZED の check を1件以上持つ table_session。卓を開けただけ (オーダーゼロ) でクローズしたセッション、全 check を VOIDED にして退店したセッション (無銭飲食)、予約 NO_SHOW (そもそも table_session が生成されない) は、客数・組数のいずれにも含まない。全額サービス (100%割引で total_jpy = 0 でも FINALIZED は実来店として含める。回収不能分は domain_event (監査) のみに記録する)。
 - 営業日の帰属 : セッション単位ではなく、紐づく check.business_date (=精算日) で日報の営業日に割り当てる。これにより「その営業日に計上される売上」と「その営業日にカウントされる客数」が同じキーで揃う。1セッションの check が締めをまたいで別営業日に分かれた場合は、そのセッションの客数・組数を 最後に FINALIZED した check の business_date に寄せる (フェーズ1では稀)。
 - 客数 (guest_count) : 上記母集団の table_session.party_size の合計。party_size は INTEGER NOT NULL のため NULL 混入はない。
 - 組数 (group_count) : 上記母集団の table_session 件数。卓の結合・分割を経た場合は、結合・分割後の最終的な table_session 単位で数える (1物理来店=最終セッション1件)。フェーズ1は会計開始後の分割を対象外にしているため実害は小さい。
 - party_size 変更のタイミング : 締め実行時点の party_size 現在値で確定する (変更履歴は持たない)。
 - 客単価 (avg_per_guest_jpy) : 分子は sales_total_jpy と同一定義 (当該 business_date の FINALIZED な check.total_jpy の合計。税込・割引後・返金控除前、VOIDED は除外)。分母は guest_count。guest_count = 0 のときは 0 を格納。端数は四捨五入。ROUNDING ディスカウントは total_jpy に反映済みのため追加処理は不要。組単価は算出しない。
 - 時間帯別 (sales_report_by_hour) : 売上額は明細・会計の時刻で複数の時間バケットに分散するが、客数はセッション単位のため分割せず、table_session.opened_at (着席時刻) の時間帯に来店客 (party_size) をまとめて計上する。

- **D2 遅延計上とのズレ**：端末が日次締めをまたいでオフラインだった結果、前営業日の来店客の一部明細が翌営業日に遅延同期（未決事項3・D2）した場合、その客数は前営業日に計上済みだが、売上は翌営業日の sales_total_jpy に載る。このズレは許容し、daily_close の再オープンや客数の遡及計上は行わない。フェーズ1では sales_daily_report に遅延計上分の内数列・フラグ列を追加せず、遅延計上額が必要になったら order_line の「前営業日からの遅延計上」フラグから集計で導出する。日報上の表示方法（内数・脚注など）はD2の決定どおり実装スパイク送りのままとする。締めをまたいで着席し続けたセッションは売上・客数とも check_business_date 側に載るため、このズレは生じない。
- 10. menu_option_# → course をフェーズ1に含めるか（現状 S）。含めない場合、order_line.note で代替。**解決済み（04_architecture.md §1決定表行10・§4.4）**：menu_option_group / menu_option / order_line_option / course をいずれも**フェーズ1に含める**。居酒屋業態でコース・飲み放題・トッピングは頻出機能であり、実装コストがテーブル追加のみで小さいため、course の中核機能（予約時のコース指定 FR-C01/C03、開始時刻記録・ラストオーダー通知 FR-E05）は既に M であり、course を落とす選択肢は実質存在しない。含める前提で以下を確定する。
 - **オプションの適用範囲（a）**：FR-D05 どおり「価格差分つきの簡易オプション」に限定する。商品（menu_item）固有のオプション群を **2階層（menu_option_group → menu_option）** だけ持ち、min_select / max_select で必須選択・上限を表現する。多段ネスト（オプションが別のオプション群を呼び、オプション単位の在庫・売り切れ（menu_option に SOLD_OUT は持たない）、オプションによる調理・KDSの分岐、条件付き価格（数量割引・組み合わせ割引）はいずれもフェーズ2以降。
 - **店舗共有オプション群（b）**：menu_option_group.menu_item_id は nullable のまま残すが、フェーズ1は**商品固有オプションのみ運用**する（実質 menu_item_id 必須）。「全ドリンク共通の水抜き」等の店舗共有オプション群はフェーズ2以降とし、当面 UI で作らせない。
 - **course と明細の関係（c）**：course はセッション単位で1つ（table_session.course_id）。会計は**コース料金1行**（course.price_jpy × 人数、または1行×数量）だけを計上し、コース構成品は **通常の order_line（単価0円）** として1品ずつ登録する。提供順の制御は 04 §9 で確定済みの fire_state（HELD / FIRED、既定 FIRED）フラグで行い、コース専用の段階（ステップ）構造は持たない——コースの逐次提供もアラカルトの「後で出す」も同じ hold/fire で処理する。各構成品の prep_type（COOK / NO_COOK）は品ごとに従来どおり効く（NO_COOK は調理KDS・kitchen_ticket の対象外）。コース構成品マスタ（course_item）はフェーズ1では持たず、スタッフが注文入力時に構成品を明細として起こす運用とする。
 - **飲み放題（type = FREE_DRINK）の会計（d）**：course.price_jpy を人数分計上する。個々のドリンクは price_jpy = 0 の order_line として記録し（注文数の可視化・在庫用）、無料明細を 実際 に生成するか否かは実装スパイクで最終化する。
 - **order_line.note の位置づけ（e）**：オプション導入後も note は残す。**価格に影響する変更 = オプション、影響しない要望（アレルギー等） = note**、と役割を分ける。
 - **ラストオーダー通知（f）**：LO時刻 = table_session.course_started_at + course.duration_min - course.last_order_before_min。LO時刻・終了時刻に notification 基盤（04 §4.10）経由でホールへ通知し、table_session.last_order_at に保持してモバイルオーダーの送信可否（mobile_order_session ACTIVE→EXPIRED）判定にも使う。FR-E05 が既に M のため項目10で新たに決めることはなく、course 採用の確定により実装可能になる、という関係。
- 11. **フェーズ2送りで確定（04_architecture.md §1決定表行11）**：本項はフェーズ2構想であり、04 では物理設計を行わない。下記の方針を維持し、フェーズ2着手時に 04 を物理設計として改訂する。
 - （フェーズ2構想）顧客の会員登録・テナント単位でのログイン・過去の予約履歴閲覧。guest（顧客）エンティティを新設し、user（スタッフ）とは別の認証経路とする想定（company_code + email の複合UKなど、命名・分離方針は user に準拠しつつ検討）。reservation には guest_id（nullable, FK→guest）を追加し、匿名予約（フェーズ1の guest_name / guest_phone / guest_email）とログイン済み顧客の予約を共存させる。

8. 次のステップ

1. 本書のレビュー（エンティティの過不足、状態値の妥当性、不変条件の合意）。
2. 04_architecture.md：物理スキーマ（DDL 方針）、API 一覧、テナント分離の実装（アプリ層強制 / RLS の要否）、決済連携の詳細、domain_event の実装方式、Mermaid 図の PDF レンダリング方針、データ保持期間の確定値。
3. フェーズ1バックログ化：エンティティ単位の CRUD と、状態遷移をまたぐユースケース（来店〜会計〜締め）をストーリー分解し、02 の FR ID に対処付ける。